

How AI Uses Language

Language Models and a Small Python Project · Lessons 5–6

AI course article 3 | Study edition

Lesson 5	Tokens, context, transformers, attention, sampling, and hallucinations.
Lesson 6	A transparent text classifier trained and tested with standard Python only.
Central idea	Probabilistic pattern recognition can produce useful language and classifications, but its outputs still require evaluation.

The previous article followed prediction errors through training and neural-network layers. This article turns to language models, then converts the main ideas into a small program that can learn from labeled messages.

Lesson 5: Language Models and Next-Token Prediction

A language model learns statistical relationships in sequences of text. It does not begin with a human-style dictionary of meanings. During training, it examines many examples and adjusts its parameters so it becomes better at predicting what is likely to come next.

Before text enters a model, it is divided into tokens. A token may be a whole word, part of a word, punctuation, or another recurring unit. Tokenization¹ allows a model to represent text as numbers that a neural network can process.

During generation, **a language model predicts one token at a time**. Given “The cat sat on the,” it might assign high probability to “mat,” “floor,” or “chair.” It selects a continuation, adds that token to the context, and repeats the process. A long response is produced through many successive predictions.

The context window is the text the model can consider at one time. Context changes which continuation is likely. After “She deposited money at the,” the word “bank” probably refers to a financial institution; after “She sat beside the river,” the same word has another likely meaning.

Modern language models commonly use a neural-network design called a transformer. Its attention mechanism helps the model compare relevant parts of the context. In a long sentence, attention can strengthen connections between words that depend on one another even when they are far apart.

The model’s output is a probability distribution, not a single fact retrieved from a perfect encyclopedia. Sampling settings influence whether generation favors the safest high-probability continuation or explores less likely alternatives. This is one reason the same prompt can produce different answers.

Training on large collections of text gives a model broad linguistic patterns, but plausible language is not the same as verified truth. A model can generate an incorrect statement that sounds confident because its immediate task is producing a likely continuation. Such an unsupported or false generation is often called a hallucination.

Quick Check

If a language model writes a grammatically excellent answer, does that prove the answer is factually correct? No. Fluency and factual reliability must be evaluated separately.

Your turn: Complete the phrase “After the storm, the street was” in three different ways. How does each continuation change the direction of the next sentence?

Lesson 6: Building a Small AI System in Python

We can demonstrate learning without a giant neural network. The accompanying Python program trains a tiny classifier to decide whether a short message belongs to the category “study” or “everyday.” Its goal is modest, but its workflow mirrors larger machine-learning projects.

First, we prepare a small labeled collection of example messages. “Review the chapter before class” is labeled study, while “Buy fruit on the way home” is labeled everyday. The labels tell the program which pattern each example should support.

Next, the program tokenizes the messages and counts how often each word appears in each category. Words such as “chapter,” “quiz,” and “notes” become evidence for study. Words such as “dinner,” “store,” and “laundry” become evidence for everyday.

The classifier uses a simple method called multinomial naive Bayes. For a new message, it combines many small pieces of evidence from the words it recognizes. Smoothing prevents an unseen word from reducing a category’s probability to zero. The category with the higher score becomes the prediction.

The program separates training examples from test examples. This matters because evaluation on new messages is essential. Testing only on the examples used for training would tell us whether the model remembered its lessons, not whether it can generalize.

You can run the project with the command “python3 tiny_text_classifier.py.” It prints predictions for several new messages and reports the held-out test accuracy. No external Python packages are required.

This model is intentionally tiny. Its dataset is too small for real deployment, its categories are simplified, and individual words can mislead it. Its value is educational: every count and score is visible, so the path from examples to prediction can be inspected.

Quick Check

Why does the program keep test examples separate from training? Because testing on unseen examples provides a more honest measure of generalization.

Your turn: Add two study messages and two everyday messages to the training data. Before running the program, predict which words will most influence the model and where it might still make mistakes.

Языковая модель предсказывает текст по одному токenu, учитывая доступный контекст. Небольшой классификатор работает похожим образом: он учится на размеченных примерах и объединяет признаки, чтобы выбрать категорию. Однако правдоподобный ответ необходимо проверять, потому что языковая плавность не гарантирует фактическую точность.²

¹ Tokenization, when applied to data security, is the process of substituting a sensitive data element with a non-sensitive equivalent, referred to as a token, that has no intrinsic or exploitable meaning or value. The token is a reference that maps back to the sensitive data through a tokenization system. The mapping from original data to a token uses methods that render tokens infeasible to reverse in the absence of the tokenization system, for example using tokens created from random numbers.

² The Russian paragraph summarizes token prediction, contextual evidence, supervised classification, and the difference between fluency and factual reliability.

Project Snapshot

Training data	8 labeled messages
Algorithm	Multinomial naive Bayes with Laplace smoothing
Dependencies	Python standard library only
Test result	4/4 held-out examples correct (100%)

```

model = NaiveBayesClassifier()
model.fit(TRAINING_DATA)
prediction = model.predict("review notes before the quiz")
print(prediction) # study

```

2nd edition

5-6 预测下一个单元 微模型

Lesson 6 note: The tiny classifier is useful because its training counts and prediction scores can be inspected directly.

Lesson 5 note: Next-token prediction is repeated generation, while context and attention influence the probability distribution at each step.

Article Vocabulary

Term	Meaning in this article
language model	A model trained to recognize and predict patterns in sequences of language.
token	A unit of text processed by a language model.
tokenization	The process of dividing text into tokens and representing them numerically.
next-token prediction	Estimating which token is likely to follow the preceding context.
context window	The amount of text a model can consider at one time.
transformer	A neural-network architecture designed to process relationships within sequences.
attention mechanism	A method that lets a model weigh relationships among parts of its context.
probability distribution	A set of possible outcomes paired with their estimated probabilities.
sampling	Choosing an output from a model's probability distribution.
hallucination	A generated claim that is unsupported or false despite sounding plausible. (幻觉)
classifier	A model that assigns an input to one of several categories.
multinomial naive Bayes	A probabilistic classifier based on feature counts within labeled categories.
smoothing	A technique preventing unseen features from forcing a probability to zero.
test accuracy	The proportion of held-out test examples classified correctly.

This course article was created by Codex, an AI coding agent from OpenAI.